

Міністерство освіти і науки України
Національний університет “Львівська політехніка”



Кафедра ЕОМ

Дослідження цифрового синтезатора частоти в МПС

Методичні вказівки

до лабораторної роботи № 6 з курсу “ Мікропроцесорні системи ”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Затверджено
на засіданні
кафедри електронних обчислювальних машин
Протокол №1 від 29.08.2019р.

Львів – 2019

Дослідження цифрового синтезатора частоти в МПС: Методичні вказівки до лабораторної роботи № 6 з курсу “Мікропроцесорні системи” для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія” / Укладач: Пуйда В.Я., – Львів: Національний університет “Львівська політехніка”, 2019, 33 с.

Укладач

Пуйда В. Я., к. т. н, доцент

Рецензент

Відповідальний за випуск:
кафедри

Мельник А. О., професор, завідувач

МЕТА РОБОТИ:

ознайомлення з технічними характеристиками, архітектурою, системою команд мікроконтролера ADuC7128/7129, з основними режимами формування сигналу цифро-аналоговим перетворювачем ADuC7128/7129 та цифровим синтезатором частоти(DDS), режимами роботи інтерфейсу SPI.

1. Перевірити свою теоретичну підготовку по контрольних питаннях для самоперевірки; при необхідності скористатися методичними матеріалами.
2. Запустити систему mVision (UVx.EXE).
3. Ознайомитись з порядком створення проекту, елементами екрану відлагоджувача, використанням команд головного меню та підменю, режимами відображення завантаженої програми, операціями модифікації вмісту комірок пам'яті та регістрів.
4. Ознайомитися з структурою ADuC7128/29, особливостями ядра ARM7, організацією пам'яті і системою команд.
5. Завантажити тестову програму, запустити покрокове виконання та проконтролювати процес формування сигналу цифро-аналоговим перетворювачем ADuC7128/29.
6. Виконати індивідуальне завдання визначене керівником лабораторної роботи.
7. Захистити лабораторну роботу.

ЗАВДАННЯ

1. В покроковому режимі продемонструвати ініціалізацію DAC та цифрового синтезатора частоти(DDS).
2. В покроковому режимі продемонструвати ініціалізацію інтерфейсу SPI
3. В покроковому режимі продемонструвати режими роботи інтерфейсу SPI
4. Продемонструвати формування сигналу цифро-аналоговим перетворювачем DAC.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Внутрішня структура мікроконтролера ADuC7128/29.
2. Структура процесорного ядра ARM7TDMI.
3. Особливості інструкцій.
4. Організація пам'яті мікроконтролера ADuC7128/29.
5. Структура DAC ADuC7128/29.
6. Порт SPI ADuC7128/29.
7. Основні етапи створення проекту та опції меню відлагоджувача.

Основні етапи виконання роботи

Завантажити відлагоджувальне середовище Keil mVision

Створення нового проекту:

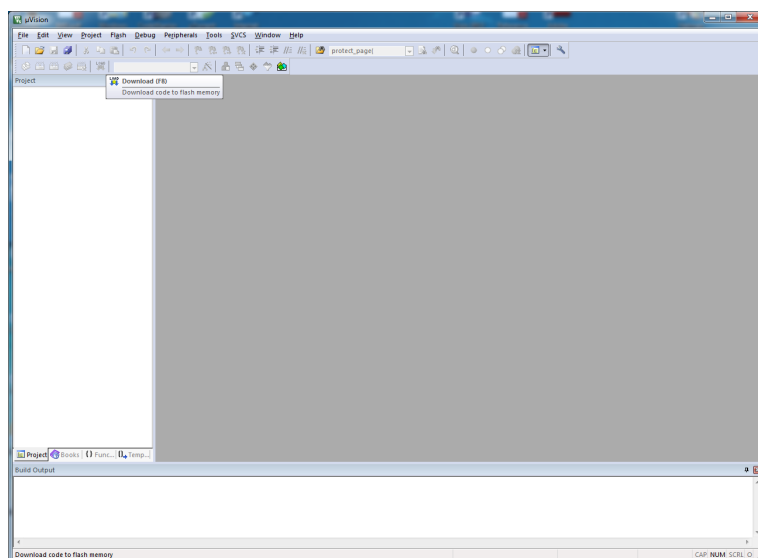


Рис.1. Стартове вікно

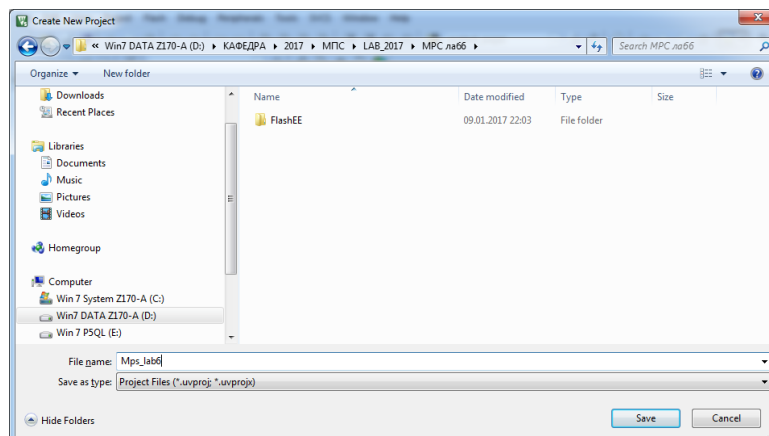


Рис.2. Створення нового проекту

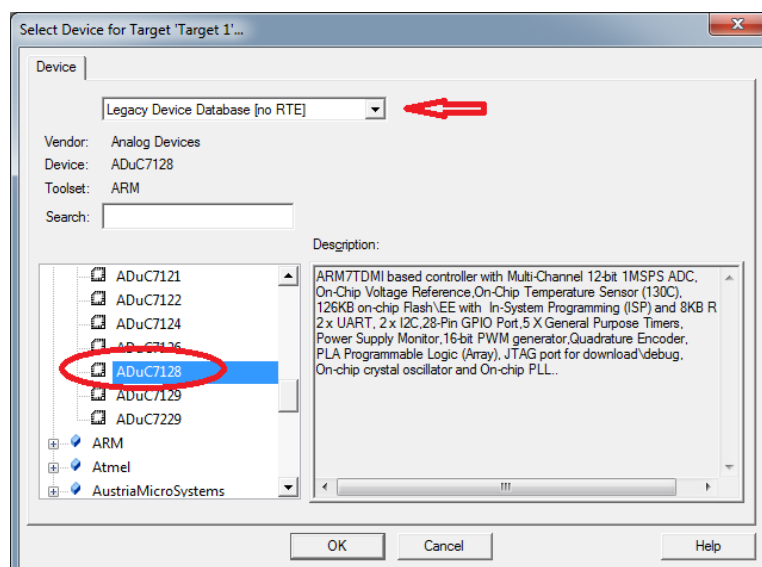


Рис.3. Вибір мікроконтролера

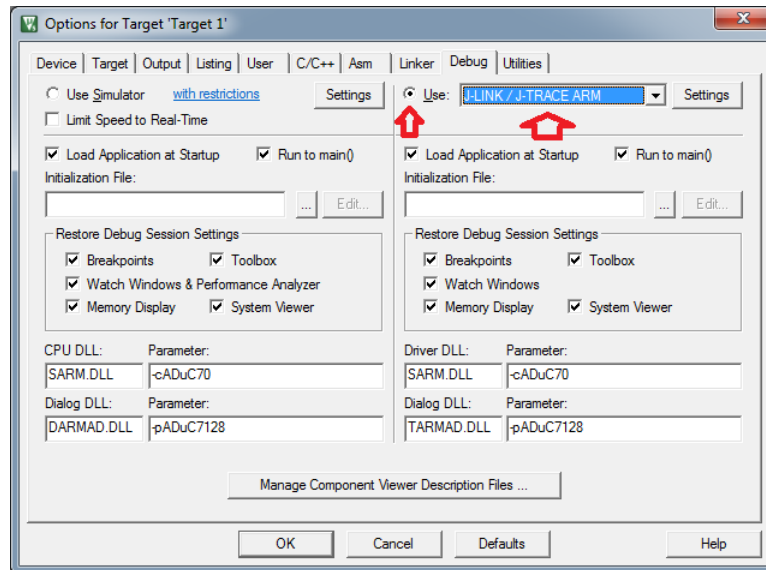


Рис.4. Вибір адаптера відлагоджувача

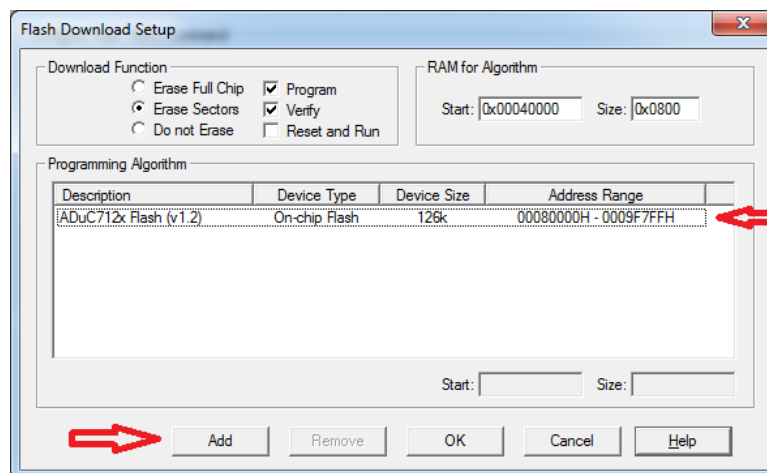


Рис.5. Встановлення конфігурації програматора Flash

Вибравши піктограму Create new file в директорії проекту створюємо файл main.c. За допомогою команди Add Existing Files to Group додаємо створений файл до проекту

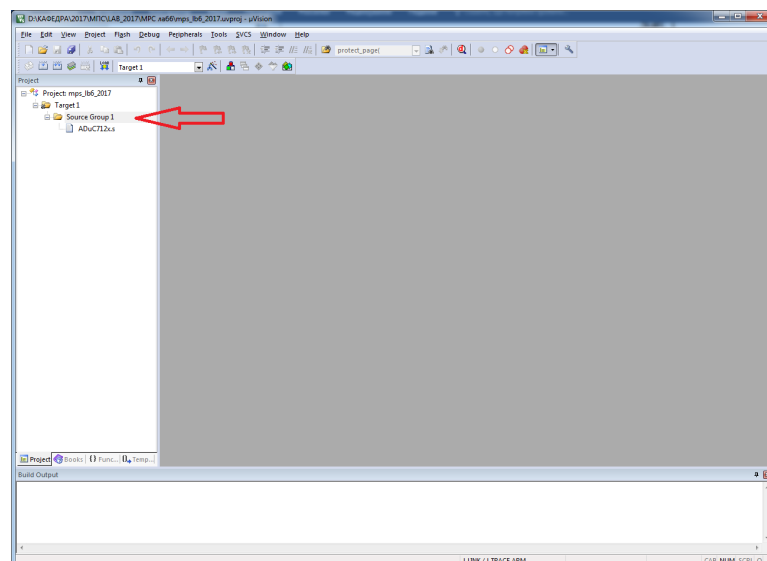


Рис.6. Add Existing Files to Group

Відкрити (Project/Open Project) тестовий проект DAC, скомпілювати проект та запустити відлагоджувач. «d»:

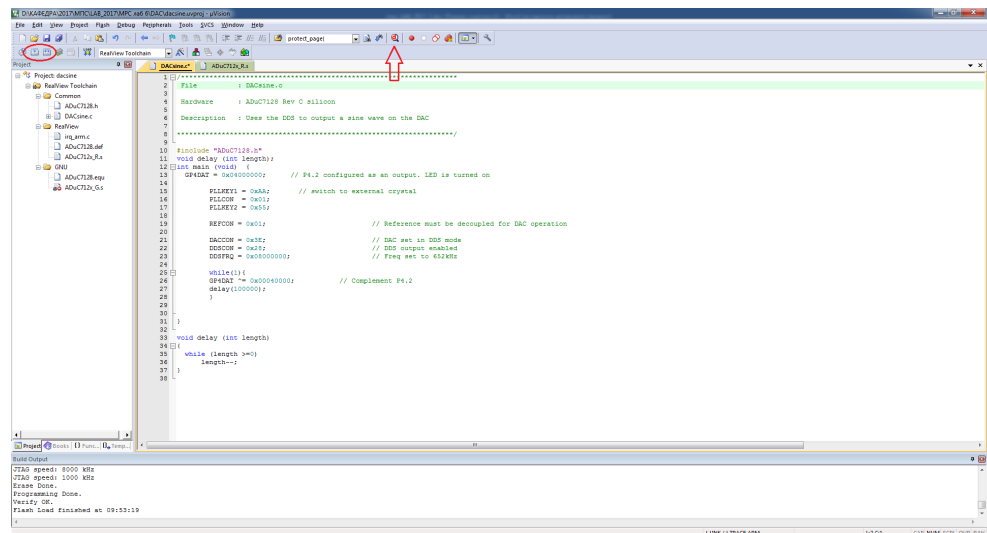


Рис.7. Компіляція проекту та запуск відлагоджувача

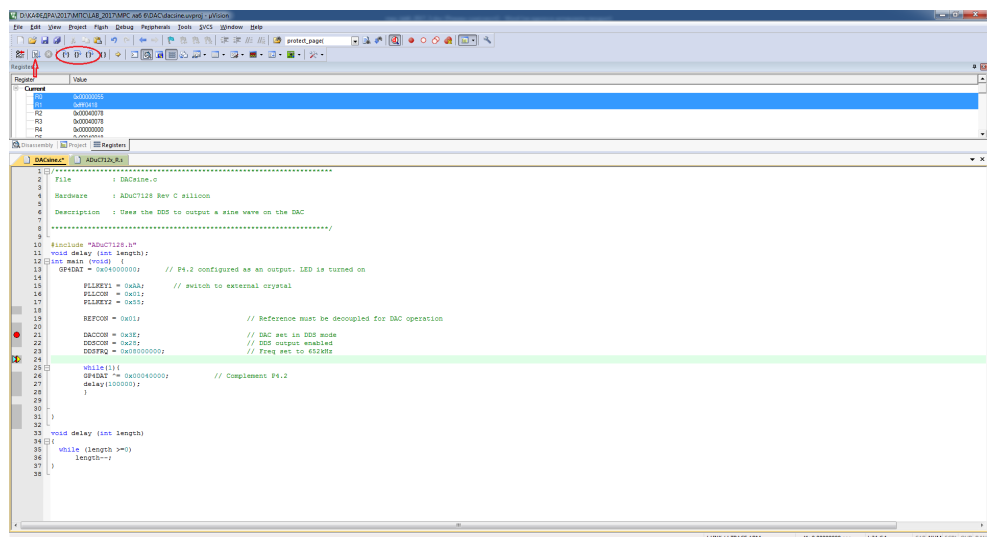


Рис.8. Вікно для відображення покрокового виконання програми

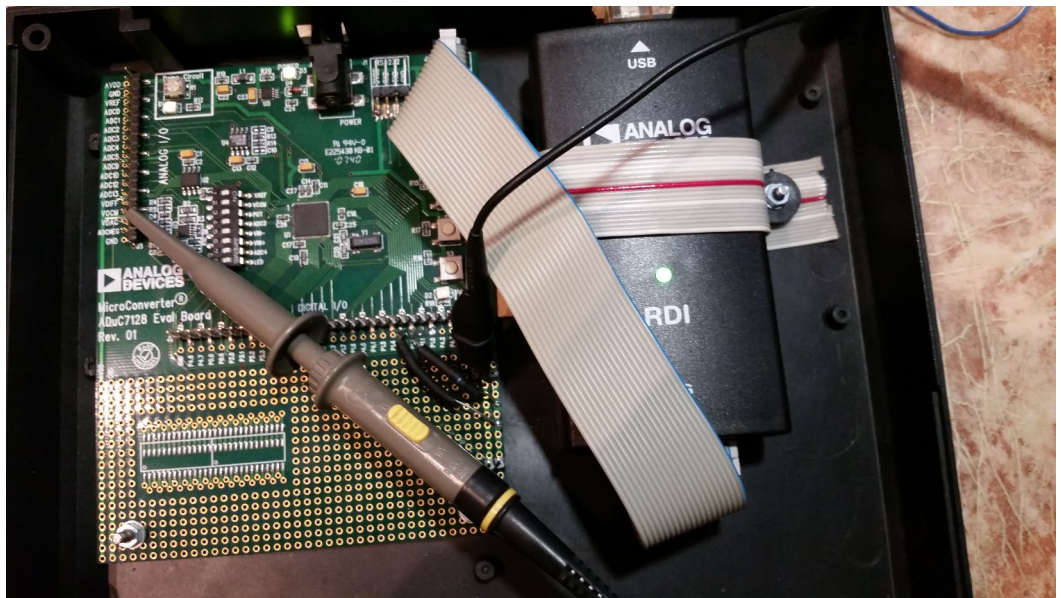
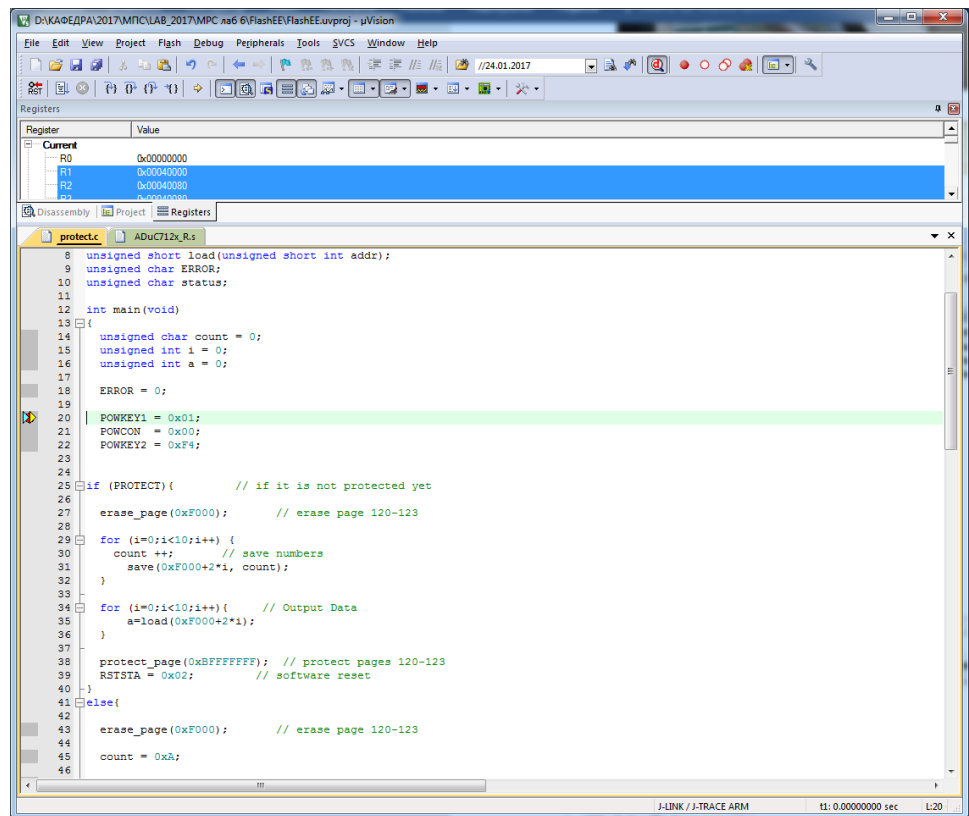


Рис.9. Відображення виходу DAC на осцилографі



МЕТОДИЧНІ МАТЕРІАЛИ

Мікроконтролер ADuC 7128/7129

Основні характеристики ADuC7128/7129:

- 44 MIPS ARM7TDMI ® ядро;
- 126 кбайт Flash на кристалі;
- 8 кбайт ОЗП (SRAM);
- АЦП, 12 розрядів, 10 каналів, 1 MSPS;
- ЦАП, 10 розрядів;
- DDS (цифровий синтезатор), 32 розряди, 21 МГц;
- прецизійний джерело опорної напруги з малим температурним дрейфом (10 стр / хв / ° C);
- послідовні інтерфейси: 2 × UART, 2 × I2C і SPI;

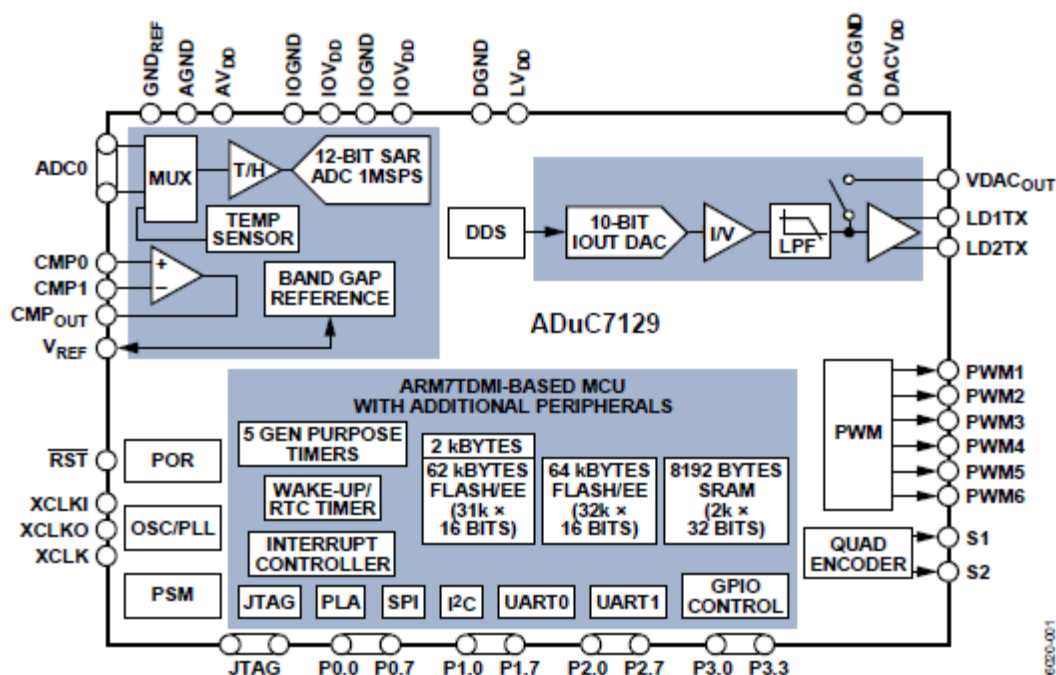


Рис.10. Внутрішня структура мікроконтролера ADuC7128/7129

Характеристика ядра ARM7TDMI

Ядро ARM7TDMI - представник сімейства ARM 32-розрядних мікропроцесорів загального призначення. Сімейство ARM характеризується високою продуктивністю при дуже низькому рівні споживання та малих розмірах.

До областей застосування ядра ARM7 відносять:

- Телекомунікації - контролери GSM терміналів.
- Обмін даними - засоби перетворення протоколів.
- Портативні обчислення - Palmtop комп'ютери.
- Портативні вимірювальні пристрої - кишенькові пристрої збору даних.
- Автомобільну техніку - пристрої керування двигуном.
- Інформаційні системи - Smart карти.
- Засоби відображення - JPEG контролери.

Архітектура ARM виконана на основі принципів RISC (комп'ютер зі скороченим набором інструкцій). Набір інструкцій RISC і пов'язаний з ним механізм дешифрування є більш простим порівняно CISC-архітектурою (комп'ютер зі складним набором інструкцій). За рахунок цього досягається:

- висока продуктивність виконання інструкцій;
- реагування на переривання в режимі реального часу
- малі розміри, ефективна вартість процесорної макрокомірки.

Основні характеристики ядра ARM7

- 32-розрядний RISC процесор (32-розрядні шини даних та адреси).
- 32-розрядна адресація - лінійне адресний простір в 4 Гб - виключає потребу в сегментованій, поділеній на банки або оверлейной пам'яті.
- Тридцять один 32-розрядний регістр загального призначення і шість регістрів стану.
- Регістри адрес, записи й конвеєра.
- Циклічний зсувний пристрій і перемножувач.
- Трирівневий конвеєр (вибірка команди, її декодування і виконання).
- Робочі режими Big Endian і Little Endian.
- Напруга живлення 3,3 і 5 В.
- Мале споживання 0,6 мА при виготовленні по CMOS технології з топологічними нормами 0,8 мкм.
- Повністю статичне робота, що дозволяє додатково знижувати споживання за рахунок зменшення тактової частоти, що ідеально для критичних застосувань.
- Швидкий відгук на переривання, що застосовується для реалізації реального масштабу часу.
- Підтримка систем віртуальної пам'яті.
- Проста але потужна система команд.

Процесор ARM7TDMI підтримує два набори інструкцій:

32-розрядний набір інструкцій ARM;

16-розрядний набір інструкцій Thumb.

Структурна, функціональна схеми процесора і процесорного ядра

На рисунку 1.2 представлені структурна схема процесора ARM7TDMI із зазначенням компонентів і основних сигналів. На рисунку 1.3 демонструється основний процесор (логіка ядра ARM7TDMI), а на рисунку 1.4 показана функціональна схема процесора ARM7TDMI з вказівкою що входять та виходять сигналів.

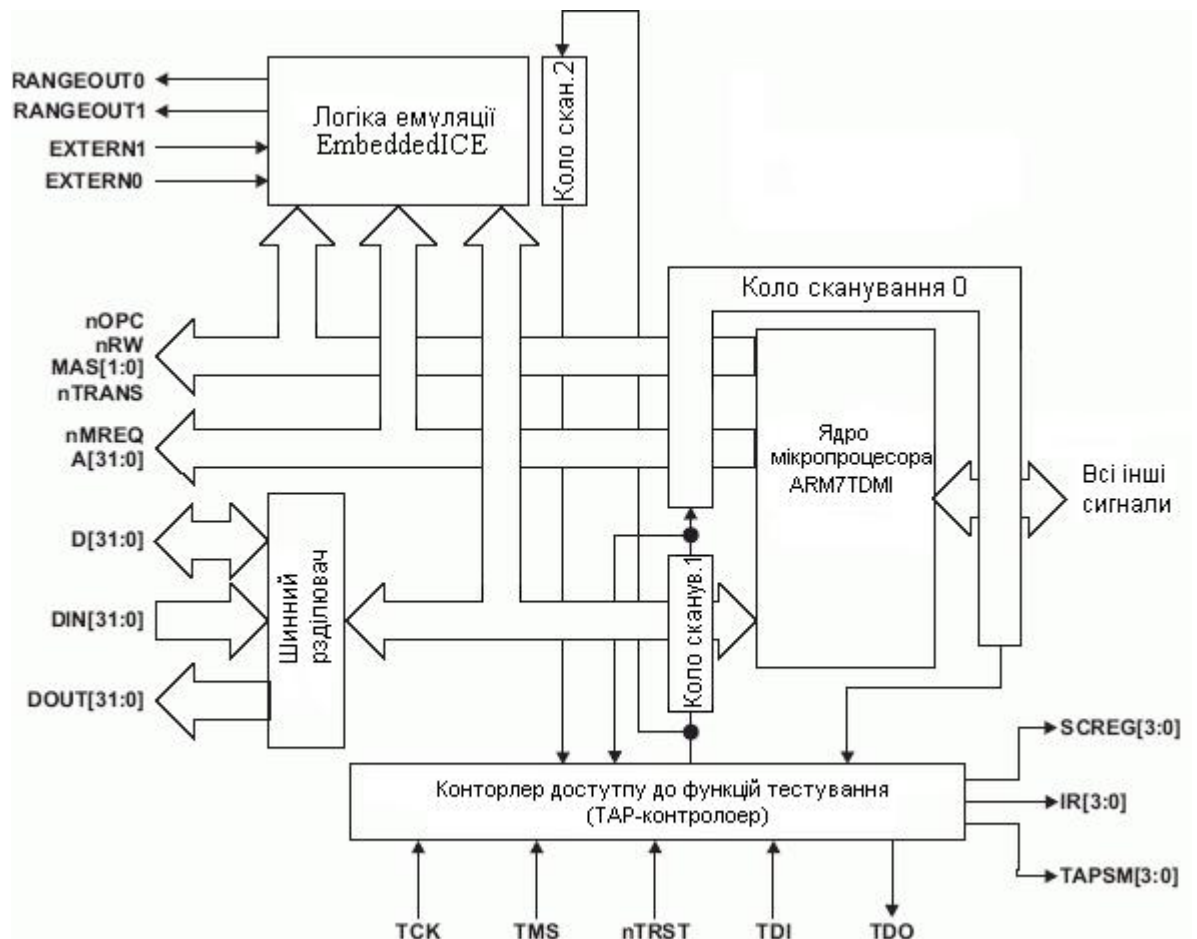


Рис. 11. Структурна схема процесора ARM7TDMI.

32-розрядна система команд ядра ARM7 містить одинадцять базових типів команд:

- Два типи використовують вбудований арифметико-логічний пристрій, циклічний зсувний пристрій і помножувач при операціях над даними в банку з 31 регістра, форматом по 32 розряду кожен;
- Три класи команд керування переміщенням даних між пам'яттю і регістрами, один оптимізований на забезпечення гнучкості адресації, інший під швидке контекстне перемикавання і третій під підкачки даних;
- Три команди керують потоком і рівнем привілегії виконання;
- Три типи призначені для управління зовнішніми сопроцесорами, що дозволяє розширити функціональні можливості системи команд за межами ядра.
- Система команд ARM добре обробляється компіляторами мов високого рівня. На відміну від деяких RISC процесорів, процесор ARM7, при виникненні необхідності в деякому зменшенні обсягу кодів, допускає програмування та на асемблері.

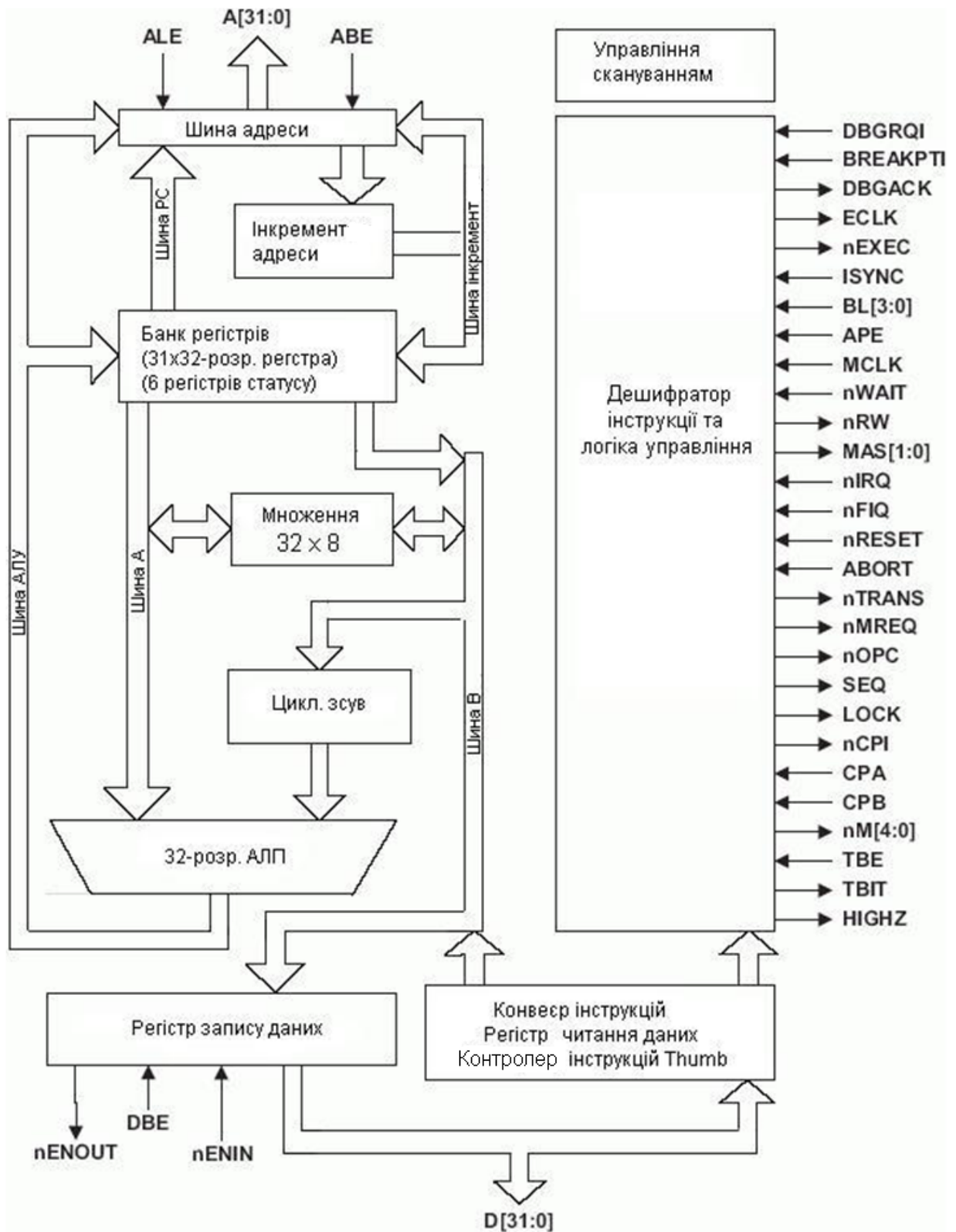


Рис. 12. Структурна схема процесорного ядра ARM7TDMI

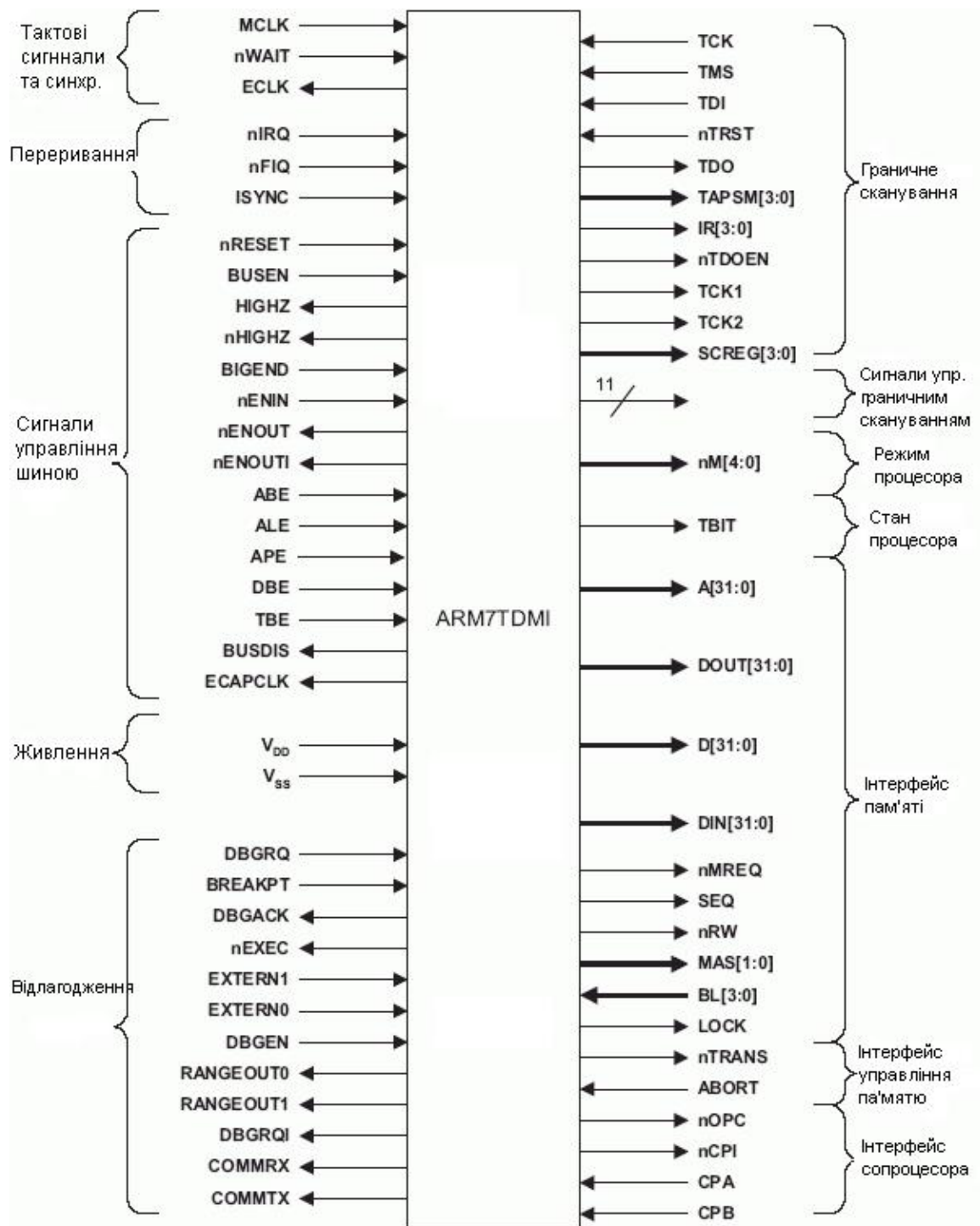


Рис. 13. Функціональна схема ядра ARM7TDMI

Організація пам'яті

ADuC7129 об'єднує три окремих блоки пам'яті : 8 kB SRAM і два 64 kB вбудованої Flash/EE пам'яті. 126 kB вбудованої Flash/EE пам'яті доступні користувачу, і тільки 2 kB зарезервовані для фабричних налаштувань завантажувальної сторінки. Розміщення цих двох блоків показано на рисунку 1.

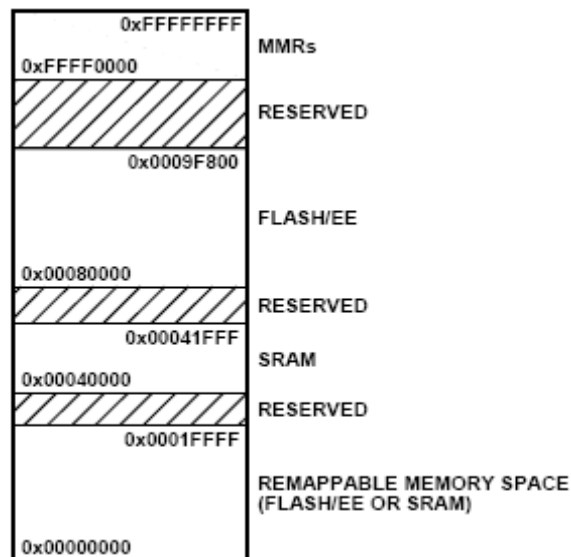


Рис.14. Організація пам'яті ADuC7128/7129

Доступ до пам'яті

ARM7 ядро бачить пам'ять як лінійну таблицю 2^{32} байт, де різні блоки пам'яті розміщені як контури на рисунку 2.

Організація пам'яті ADuC7129 сконфігурована за принципом молодший вперед: найменш значущий байт розміщений в молодшому байті адреси, а найбільш значущий – в старшому байті адреси.

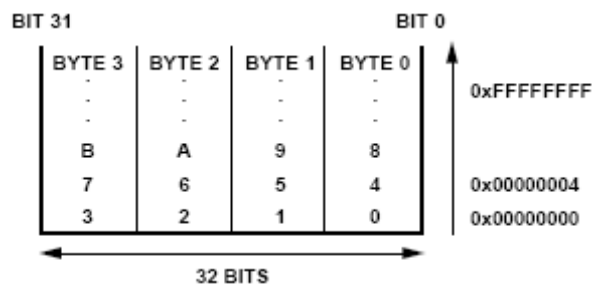


Рис.15. Структура комірки пам'яті

FLASH/EE Пам'ять

128 kB Flash/EE організовано як два блоки 32 k x 16 біт. В першому блоці, 31 k x 16 біт є пам'яттю користувача і 1 k x 16 біт зарезервовані для фабричних налаштувань завантажувальної сторінки. Розмір сторінки даної Flash/EE пам'яті є 512 байт.

Другий блок 64 kB організовано схожим чином. Він впорядкований як 32 k x 16 біт. Всі вони доступні користувачу.

126 kB Flash/EE є доступними користувачу як пам'ять програм та пам'ять констант. Різниця між пам'яттю даних і пам'яттю програм немає, оскільки ARM програма розподіляє між ними спільний простір. Ширина Flash/EE пам'яті є 16 біт, мається на увазі, що в ARM режимі (32-бітної інструкції), подвійний доступ до Flash/EE є необхідним для витягнення інструкції з пам'яті. Тому для оптимального часу доступу до пам'яті рекомендується використовувати Thumb режим. Максимальна частота роботи Flash/EE пам'яті - 41.78 MHz в Thumb режимі і 20.89 MHz в ARM режимі.

SRAM

8 kB SRAM є доступними користувачу, організовані як 2 k x 32 біти, це є 2 k слів. Оскільки пам'ять SRAM має ширину 32-біт, то ARM програма може запускатись прямо з SRAM на частоті 41.78 MHz.

Набір інструкцій ядра ARM7TDMI.

Набір інструкцій ARM представлений в таблиці 1.1.

Таблиця 1.1.

Преставлення інструкцій ARM

Операції		Синтаксис Ассемблера
Пересылка	Пересилання	MOV {cond}{S} Rd, <Oprnd2>
	Пересилання NOT	MVN {cond}{S} Rd, <Oprnd2>
	Пересилання SPSR в реєстр	MRS {cond} Rd, SPSR
	Пересилання CPSR в реєстр	MRS {cond} Rd, CPSR
	Пересилання реєстра SPSR	MSR {cond} SPSR{field}, Rm
	Пересилання CPSR	MSR {cond} CPSR{field}, Rm
	Пересилання константи у прапори SPSR	MSR {cond} SPSR_f, #32bit_Imm
	Пересилання константи у прапори CPSR	MSR {cond} CPSR_f, #32bit_Imm
Арифметичні	Додавання	ADD {cond}{S} Rd, Rn, <Oprnd2>
	Додавання з переносом	ADC {cond}{S} Rd, Rn, <Oprnd2>
	Віднімання	SUB {cond}{S} Rd, Rn, <Oprnd2>
	Віднімання з переносом	SBC {cond}{S} Rd, Rn, <Oprnd2>
	Віднімання зворотнього віднімання	RSB {cond}{S} Rd, Rn, <Oprnd2>
	Віднімання зворотнього віднімання з переносом	RSC {cond}{S} Rd, Rn, <Oprnd2>
	Множення	MUL {cond}{S} Rd, Rm, Rs
	Множення-накопичення	MLA {cond}{S} Rd, Rm, Rs, Rn
	Множення длинных беззнаковых чисел	UMULL {cond}{S} RdLo, RdHi, Rm, Rs
	Множення - беззнакове накопичення значень	UMLAL {cond}{S} RdLo, RdHi, Rm, Rs
	Множення знакових даних	SMULL {cond}{S} RdLo, RdHi, Rm, Rs
	Множення – знакове накопичення значень	SMLAL {cond}{S} RdLo, RdHi, Rm, Rs
	Порівняння	CMP {cond} Rd, <Oprnd2>
	Порівняння негативне	CMN {cond} Rd, <Oprnd2>
Логічні	Перевірка	TST {cond} Rn, <Oprnd2>
	Перевірка на еквівалентність	TEQ {cond} Rn, <Oprnd2>
	Лог. І	AND {cond}{S} Rd, Rn, <Oprnd2>
	Виключне АБО	EOR {cond}{S} Rd, Rn, <Oprnd2>
	ORR	ORR {cond}{S} Rd, Rn, <Oprnd2>
	Сброс бита	BIC {cond}{S} Rd, Rn, <Oprnd2>>
Переход	Перехід	B {cond} label
	Перехід за посиланням	BL {cond} label
	Перехід і зміні набору інструкцій	BX {cond} Rn
Чтение	Читання слова	LDR {cond} Rd, <a_mode2>
	Читання слова з перевагою режиму користувача	LDR {cond}T Rd, <a_mode2P>
	Читання байта	LDR {cond}B Rd, <a_mode2>
	Читання байта з перевагою режиму користувача	LDR {cond}BT Rd, <a_mode2P>
	Читання байта зі знаком	LDR {cond}SB Rd, <a_mode3>
	Читання напівслова	LDR {cond}H Rd, <a_mode3>
	Читання напівслова зі знаком	LDR {cond}SH Rd, <a_mode3>
	Читання операции с декількома блоками даних	-
	• з попереднім інкрементом	LDM {cond}IB Rd{!}, <reglist>{^}

	• з подальшим інкрементом	LDM {cond}IA Rd{!}, <reglist>{^}
	• с предварительным декрементом	LDM {cond}DB Rd{!}, <reglist>{^}
	• з попереднім декрементом	LDM {cond}DA Rd{!}, <reglist>{^}
	• операція над стеком	LDM {cond}<a_mode4L> Rd{!}, <reglist>
	• операція над стеком і відновлення CPSR	LDM {cond}<a_mode4L> Rd{!}, <reglist+pc>^
	операція над стеком з регістрами користувача	LDM {cond}<a_mode4L> Rd{!}, <reglist>^
Запис	Запис слова	STR {cond} Rd, <a_mode2>
	Запис слова з перевагою режиму користувача	STR {cond}T Rd, <a_mode2P>
	Запис байта	STR {cond}B Rd, <a_mode2>
	Запис байта з перевагою режиму користувача	STR {cond}BT Rd, <a_mode2P>
	Запис полуслова	STR {cond}H Rd, <a_mode3>
	Запис операції над декількома блоками даних	-
	• з попереднім інкрементом	STM {cond}IB Rd{!}, <reglist>{^}
	• з наступним інкрементом	STM {cond}IA Rd{!}, <reglist>{^}
	• з попереднім декрементом	STM {cond}DB Rd{!}, <reglist>{^}
	• з наступним декрементом	STM {cond}DA Rd{!}, <reglist>{^}
	• операція над стеком	STM {cond}<a_mode4S> Rd{!}, <reglist>
	• операція над стеком з регістрами користувача	STM {cond}<a_mode4S> Rd{!}, <reglist>^
Обмін	слів	SWP {cond} Rd, Rm, [Rn]
	байт	SWP {cond}B Rd, Rm, [Rn]
Сопроцессор	Операція над байтами	CDP {cond} p<cpnum>, <op1>, CRd, CRn, CRm, <op2>
	Пересилання в ARM-регістр з сопроцесора	MRC {cond} p<cpnum>, <op1>, Rd, CRn, CRm, <op2>
	Пересилання в сопроцесор з ARM-регістра	MCR {cond} p<cpnum>, <op1>, Rd, CRn, CRm, <op2>
	Читання	LDC {cond} p<cpnum>, CRd, <a_mode5>
	Запис	STC {cond} p<cpnum>, CRd, <a_mode5>
Програмне переривання		SWI 24bit_Imm

Стиснення інструкцій

Мікропроцесорні архітектури, як правило, використовують набір інструкцій тієї ж розрядності, що і дані. Отже, 32-розрядні архітектури володіють поліпшеними характеристиками обробки 32-розрядних даних і можуть більш ефективно адресувати великі адресні простору в порівнянні з 16-розрядними архітектурою. 16-розрядні архітектури в більшості випадків мають більш високою щільністю коду, але приблизно вдвічі поступаються за продуктивністю. Thumb реалізує 16-розрядний набір інструкцій у складі 32-розрядної архітектури, який забезпечує:

- більш високу продуктивність у порівнянні з 16-розрядної архітектурою
- більш високу щільність коду в порівнянні з 32-розрядної архітектурою.

Набір інструкцій Thumb

Набір інструкцій Thumb є вибіркою найбільш часто використовуваних 32-розрядних інструкцій ARM. Thumb-інструкції характеризуються розміром 16 біт та мають відповідну 32-розрядну інструкцію ARM. Відповідні інструкції надають однаковий вплив на модель

процесора. Thumb-інструкції працюють зі стандартною конфігурацією ARM-реєстру, забезпечуючи чудову взаємодія між станами ARM і Thumb.

У процесі виконання 16-розрядна інструкція піддається декомпресії в реальному часі до повних 32-розрядних інструкцій ARM без втрати продуктивності.

Thumb успадковує всі переваги 32-розрядного ядра:

- 32-розрядне адресний простір
- 32-розрядні реєстри
- 32-розрядне зсувне пристрій і арифметико-логічного пристрій (АЛП)
- Пересилання в пам'ять 32-розрядних даних.

Отже, інструкції Thumb характеризуються великим діапазоном переходу, потужними арифметичними операціями і великим простором.

Код Thumb зазвичай займає 65% від ARM-коду і досягає 160% продуктивності ARM-коду при виконанні з 16-розрядної системи пам'яті. Отже, Thumb, робить ядро ARM7TDMI ідеально підходящим під вбудовані додатки, де в якості критичних параметрів виступають щільність коду і габарити.

Доступність обох наборів інструкцій, 16-розрядного Thumb і 32-розрядного ARM, дає розробникам гнучкість по оптимізації швидкодії або розміру коду на рівні процедури відповідно до вимог до їх програми. Наприклад, критичні цикли у таких програмах, як обробка часто виникають переривань і алгоритми цифрової обробки сигналів, можуть кодуватися за допомогою повних ARM-інструкцій, а потім лінкована Thumb-кодом.

Конвеєр інструкцій

Для прискорення потоку інструкцій, що надходить до процесора, в ядрі ARM7TDMI використовується конвеєр. Він дозволяє виконувати кілька операцій одночасно, а також забезпечує сталість роботи системи пам'яті і обробки. Використовується триступінчат конвейеризації, тому що інструкції виконуються в три етапи:

- Вибірка
- Дешифрування
- Виконання.

Конвеєр інструкцій показаний на рисунку .

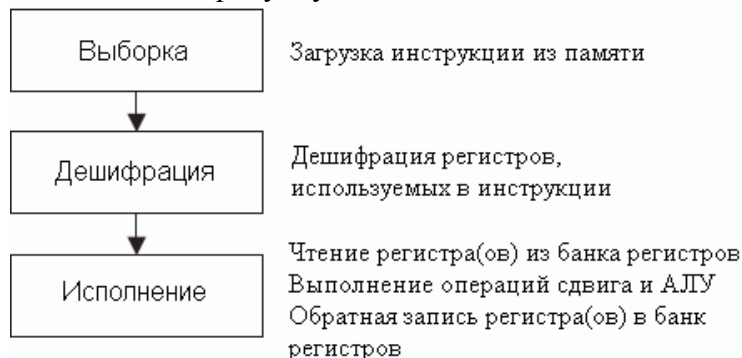


Рис. 16. Конвеєр інструкцій.

У процесі нормальної роботи, коли виконується одна інструкція, наступна інструкція декодується, а третій - зчитується з пам'яті. Лічильник програми вказує на що завантажується з пам'яті інструкцію, а не на виконувану інструкцію. Це важливо знати, тому що значення лічильника програми (PC), що використовується при виконанні інструкції, завжди на дві інструкції випереджає адресу.

Доступ до пам'яті

Ядро ARM7TDMI використовує фон-неймановскую архітектуру доступу до пам'яті з одною 32-розрядною шиною даних, по якій передаються як дії, так і дані. Доступ до даних до пам'яті можуть здійснювати тільки інструкції читання, запису та обміну. В якості даних можуть виступати:

8 біт (байт);
16 біт (півслова);
32 біт (слово).

Слова повинні бути вирівняні до 4-байтним кордонів, а півслова - до 2-байтним.

Інтерфейс пам'яті

При розробці інтерфейсу пам'яті процесора ARM7TDMI стояло завдання максимально повністю використовувати потенціал за швидкодією і при цьому мінімізувати використання пам'яті. Щоб забезпечити можливість реалізації функцій системного управління на основі стандартної малопотужною логіки критичні до швидкодії сигнали управління були конвейерізовані. Дані сигнали управління спрощують використання швидкодіючих пакетних режимів передачі, які використовуються більшістю вбудованої і зовнішньої технологій пам'яті.

Ядро ARM7TDMI підтримує чотири основних цикли доступу до пам'яті:

холостий цикл
непослідовний цикл
послідовний цикл
цикл передачі регістра співпроцесора.

Логіка EmbeddedICE

Логіка EmbeddedICE - допоміжний апаратний блок, який робить ARM-процесори налагоджували і істотно полегшує процес налагодження. Він дозволяє за допомогою програмних інструментальних засобів налагоджувати програмний код, який виконується цільовим процесором. Логіка EmbeddedICE управляється через порт JTAG з використанням інтерфейсу EmbeddedICE.

Режими адресації

Режими адресації - процедури, які використовуються різними інструкціями для генерації значень, що використовуються інструкціями. Процесор ARM7TDMI підтримує 5 режимів адресації:

Режим 1 - зсувне операнди інструкцій для обробки даних.
Режим 2 - Читання і запис слова або беззнакового байта.
Режим 3 - Читання і запис півслова або завантаження знакового байта.
Режим 4 - Множинні читання і запис.
Режим 5 - Читання і запис співпроцесора.

Режими

роботи

Процесор ARM7TDMI підтримує сім режимів роботи:

1. Режим користувача - звичайний стан ARM при виконанні програми, також використовується для виконання більшості прикладних програм.
2. Режим швидкого переривання (FIQ), який підтримує передачу даних або обробку каналу.
3. Режим переривання (IRQ), який використовується для обробки переривань загального призначення.
4. Супервізорний режим, який є захищеним режимом для операційної системи.
5. Аварійний режим, який вводиться після аварійної вибірки даних або інструкції.
6. Системний режим - привілейований режим користувача для операційної системи.

Прим.: Ви можете вводити системний режим з іншого привілейованого режиму тільки шляхом зміни біта режиму в регістрі поточного стану програми (CPSR).

7. Невизначений режим вводиться, коли виконується невизначена інструкція.

Всі режими, окрім режиму користувача, спільно називаються привілейованими режимами. Привілейовані режими використовуються для обслуговування переривань і виняткових ситуацій, а також для доступу до захищених ресурсів.

Регістри

Процесор ARM7TDMI містить усього 37 регістрів:

- 31 32-розрядних регістра загального призначення
- 6 регістрів статусу.

Не всі регістри доступні в один і той же час. Доступність регістрів для програміста залежить від стану процесора і робочого режиму.

Набір регістрів в режимі ARM

У режимі ARM доступні 16 регістрів загального призначення, один або два регістра статусу. У привілейованих режимах стають доступними специфічні банки регістрів. На рис. 2.3 демонструється, які регістри доступні в кожному режимі. У набір регістрів в режимі ARM входять 16 регістрів r0 ... r15. Ще один регістр, CPSR, містить прапори умови коду та біти поточного режиму. Регістри r0 ... r13 є регістрами загального призначення і можуть використовуватися як для зберігання даних, так і для зберігання адреси.

Регістри r14 та r15 виконують наступні спеціальні функції:

Регістр зв'язку

Регістр 14 використовується як регістр зв'язку (LR) підпрограми. Регістр r14 приймає копію регістра r15 при виконанні інструкції перехід по посиланню (BL). У всіх інших випадках регістр r14 може використовуватися як регістр загального призначення. Відповідні банкіровані регістри r14_svc, r14_irq, r14_fiq, r14_abt і r14_und аналогічним чином використовуються для запам'ятовування значень повернення r15 при виникненні переривань і виняткових ситуацій або при виконанні інструкції BL усередині процедур обробки переривань або виняткових ситуацій.

Лічильник програми

Регістр 15 зберігає значення PC. У режимі ARM біти [1:0] регістра r15 мають невизначений значення і повинні ігноруватися. Біти [31:2] містять значення PC.

У стані Thumb біт [0] має невизначене значення і повинен ігноруватися. Біти [31:1] містять значення PC. Регістр r13 використовується як показник стека (SP).

У привілейованих режимах доступний ще один регістр - регістр зберігання статусу програми (SPSR). Він містить прапори коду умови й біти режиму, що є результатом виняткової ситуації, яке викликало входження в поточний режим. Банки регістрів - дискретні фізичні регістри в ядрі, які знаходяться в позиціях доступних регістрів в залежності від поточного робочого режиму процесора. Вміст банкірованого регістра запам'ятовується при змінах робочих режимів.

У режимі FIQ є сім банкірованих регістрів в позиціях r8-r14 (r8_fiq-r14_fiq).

У стані ARM кілька обробників швидких переривань (FIQ) не повинні виконувати запис в будь-якій регістр.

У режимах користувача, IRQ, супервізорном, аварійному та невизначеному є два банкірованих регістра в позиції r13 та r14, дозволяючи зберігати власне значення SP і LR в кожному режимі.

У системному режимі використовуються ті ж регістри, що і в режимі користувача.

Регістри загального призначення і лічильник програми в режимі ARM					
Системний режим та режим користувача	Режим швидкого перевикання	Супервізорний режим	Аварійний режим	Режим перевикання	Невизначений режим
r0	r0	r0	r0	r0	r0
r1	r1	r1	r1	r1	r1
r2	r2	r2	r2	r2	r2
r3	r3	r3	r3	r3	r3
r4	r4	r4	r4	r4	r4
r5	r5	r5	r5	r5	r5
r6	r6	r6	r6	r6	r6
r7	r7	r7	r7	r7	r7
r8	r8_fiq	r8	r8	r8	r8
r9	r9_fiq	r9	r9	r9	r9
r10	r10_fiq	r10	r10	r10	r10
r11	r11_fiq	r11	r11	r11	r11
r12	r12_fiq	r12	r12	r12	r12
r13	r13_fiq	r13_svc	r13_abt	r13_irq	r13_und
r14	r14_fiq	r14_svc	r14_abt	r14_irq	r14_und
r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)	r15 (PC)

Регістри статусу програм в режимі ARM					
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_fiq	SPSR_svc	SPSR_abt	SPSR_irq	SPSR_und

Рис. 17. Організація регістрів в режимі ARM

Цифро-аналоговий перетворювач (DAC) та цифровий синтезатор частоти(DDS)

Мікроконтролери ADuC7128/ADuC7129 мають вбудований 10-бітний цифро-аналоговий перетворювач (DAC), який може використовуватися для формування заданих користувачем сигналів різної форми або синусоїдальних сигналів, що генеруються цифровим синтезатором частоти (DDS). Сигнал DAC формується з 10-бітним струмовим IDAC з подальшим перетворенням струму в напругу. На виході з IDAC є RC-фільтр, який перетворює струм в напругу. Ця напруга поступає на операційний підсилювач (ОП) з якого поступає на вихідну лінію. Для функціонування DAC використовується внутрішнє джерело опорної напруги 2,5В, яке включається встановленням REFCON=0x01.

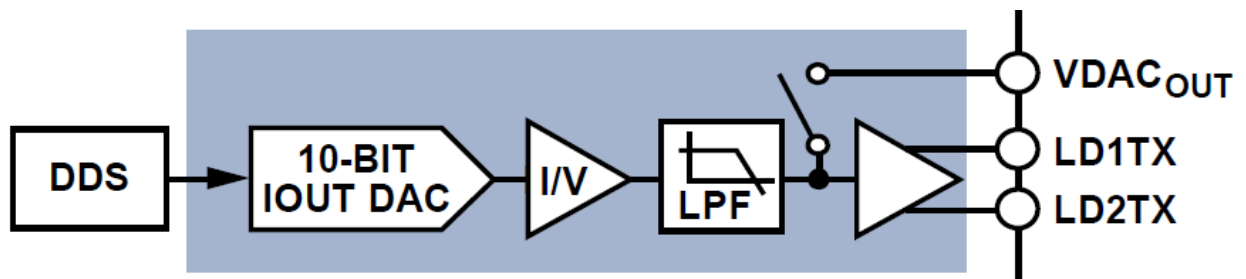


Рис. 18. Внутрішня структура DAC

0xFFFF06BC	DDS DAC	Address	Name	Byte	Access Type
0xFFFF0690		0x0670	DACCON	2	R/W
		0x0690	DDSCON	1	R/W
		0x0694	DDSFQ	4	R/W
0xFFFF0688		0x0698	DDSPHS	2	R/W
0xFFFF0670		0x06A4	DACKEY0	1	R/W
		0x06B4	DACDAT	2	R/W
		0x06B8	DACEN	1	R/W
		0x06BC	DACKEY1	1	R/W

Рис. 19. Регістри DDS та DAC

Регістри DAC:

Структура регістра DACCON

Bit	Value	Description
10:9		Reserved. These bits should be written to 0 by the user.
8		Reserved. This bit should be written to 0 by the user.
7		Reserved. This bit should be written to 0 by the user.
6		Reserved. This bit should be written to 0 by the user.
5		Output Enable. This bit operates in all modes. In Line Driver mode, this bit should be set. Set by user to enable the line driver output. Cleared by user to disable the line driver output. In this mode the line driver output is high impedance.
4		Single-Ended or Differential Output Control. Set by user to operate in differential mode, the output is the differential voltage between LD1TX and LD2TX. The voltage output range is $V_{REF}/2 \pm V_{REF}/2$. Cleared by user to reference the LD1TX output to AGND. The voltage output range is $AV_{DD}/2 \pm V_{REF}/2$.
3		Reserved. This bit should be set to 0 by the user.
2:1		Operation Mode Control. This bit selects the mode of operation of the DAC.
	00	Power-Down.
	01	Reserved.
	10	Reserved.
	11	DDS and DAC Mode. Selected by DACEN.
0		DAC Update Rate Control. This bit has no effect when in DDS mode. Set by user to update the DAC on the negative edge of Timer1. This allows the user to use any one of the core CLK, OSC CLK, baud CLK, or user CLK and divide these down by 1, 16, 256, or 32,768. A user can do waveform generation by writing to the DAC data register from RAM and updating the DAC at regular intervals via Timer1. Cleared by user to update the DAC on the negative edge of HCLK.

Name	Address	Default Value	Access
DACEN	0xFFFF06B8	0x00	R/W
Bit	Description		
7:1	Reserved.		
0	Set to 1 by the user to enable DAC mode. Set to 0 by the user to enable DDS mode.		

Name	Address	Default Value	Access
DACDAT	0xFFFF06B4	0x0000	R/W
Bit	Description		
15:10	Reserved.		
9:0	10-bit data for DAC.		

DACEN	DACDAT
DACKEY0 = 0x07	DACKEY0 = 0x07
DACEN = user value	DACDAT = user value
DACKEY1 = 0xB9	DACKEY1 = 0xB9

Перістри DDS:

Структура перістра DDSCON

Bit	Description																				
7:6	Reserved.																				
5	DDS Output Enable. Set by user to enable the DDS output. This has an effect only if the DDS is selected in DACCON. Cleared by user to disable the DDS output.																				
4	Reserved.																				
3:0	Binary Divide Control.																				
	<table> <tr> <th>DIV</th><th>Scale Ratio</th></tr> <tr><td>0000</td><td>0.000</td></tr> <tr><td>0001</td><td>0.125</td></tr> <tr><td>0010</td><td>0.250</td></tr> <tr><td>0011</td><td>0.375</td></tr> <tr><td>0100</td><td>0.500</td></tr> <tr><td>0101</td><td>0.625</td></tr> <tr><td>0110</td><td>0.750</td></tr> <tr><td>0111</td><td>0.875</td></tr> <tr><td>1xxx</td><td>1.000</td></tr> </table>	DIV	Scale Ratio	0000	0.000	0001	0.125	0010	0.250	0011	0.375	0100	0.500	0101	0.625	0110	0.750	0111	0.875	1xxx	1.000
DIV	Scale Ratio																				
0000	0.000																				
0001	0.125																				
0010	0.250																				
0011	0.375																				
0100	0.500																				
0101	0.625																				
0110	0.750																				
0111	0.875																				
1xxx	1.000																				

Name	Address	Default Value	Access
DDSFRQ	0xFFFF0694	0x00000000	R/W

Bit	Description
31:0	Frequency select word (FSW)

$$Frequency = \frac{FSW \times 20.8896 \text{ MHz}}{2^{32}}$$

Name	Address	Default Value	Access
DDSPHS	0xFFFF0698	0x00000000	R/W

Bit	Description
31:12	Reserved
11:0	Phase

$$Phase \text{ Offset} = \frac{2 \times \pi \times Phase}{2^{12}}$$

SPI - SERIAL PERIPHERAL INTERFACE

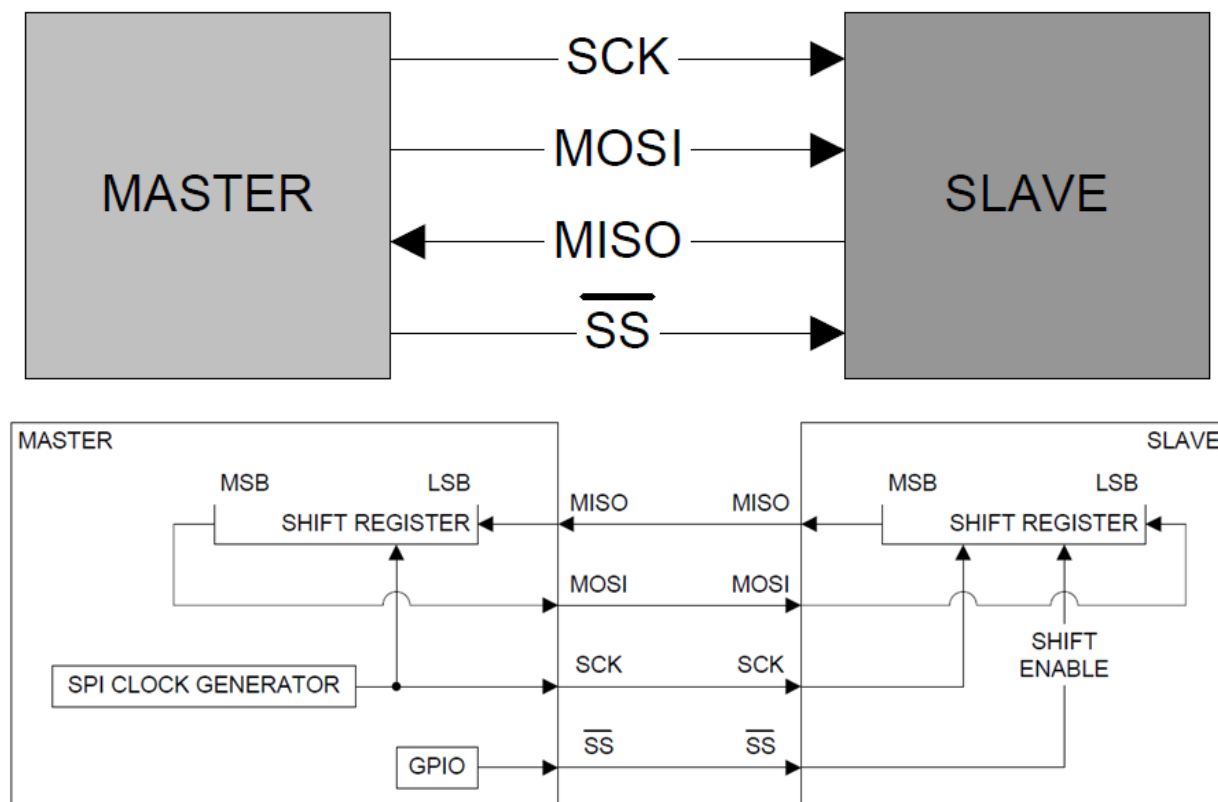
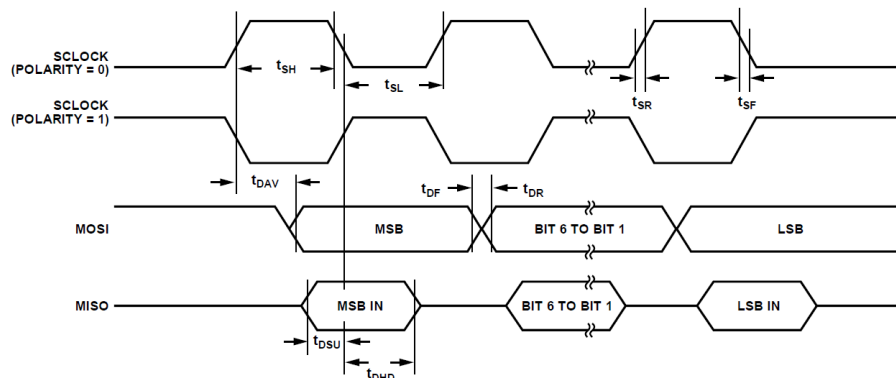


Рис. 20. Структура узла SPI

SPI Master Mode Timing (PHASE Mode = 1)					
Parameter	Description	Min	Typ	Max	Unit
t_{SL}	SCLOCK low pulse width ¹		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{SH}	SCLOCK high pulse width ¹		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{DAV}	Data output valid after SCLOCK edge			$2 \times t_{HCLK} + 2 \times t_{UCLK}$	ns
t_{DSU}	Data input setup time before SCLOCK edge ²	$1 \times t_{UCLK}$			ns
t_{DHD}	Data input hold time after SCLOCK edge ²	$2 \times t_{UCLK}$			ns
t_{DF}	Data output fall time		5	12.5	ns
t_{DR}	Data output rise time		5	12.5	ns
t_{SR}	SCLOCK rise time		5	12.5	ns
t_{SF}	SCLOCK fall time		5	12.5	ns

¹ t_{HCLK} depends on the clock divider or CD bits in the PLLCON MMR, $t_{HCLK} = t_{UCLK}/2^{CD}$.

² $t_{UCLK} = 23.9$ ns. It corresponds to the 41.78 MHz internal clock from the PLL before the clock divider.



SPI Master Mode Timing (PHASE Mode = 0)					
Parameter	Description	Min	Typ	Max	Unit
t_{SL}	SCLOCK low pulse width ¹		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{SH}	SCLOCK high pulse width ¹		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{DAV}	Data output valid after SCLOCK edge			$2 \times t_{HCLK} + 2 \times t_{UCLK}$	ns
t_{DOSU}	Data output setup before SCLOCK edge			75	ns
t_{DSU}	Data input setup time before SCLOCK edge ²	$1 \times t_{UCLK}$			ns
t_{DHD}	Data input hold time after SCLOCK edge ²	$2 \times t_{UCLK}$			ns
t_{DF}	Data output fall time		5	12.5	ns
t_{DR}	Data output rise time		5	12.5	ns
t_{SR}	SCLOCK rise time		5	12.5	ns
t_{SF}	SCLOCK fall time		5	12.5	ns

¹ t_{HCLK} depends on the clock divider or CD bits in the PLLCON MMR, $t_{HCLK} = t_{UCLK}/2^{CD}$.

² $t_{UCLK} = 23.9$ ns. It corresponds to the 41.78 MHz internal clock from the PLL before the clock divider.

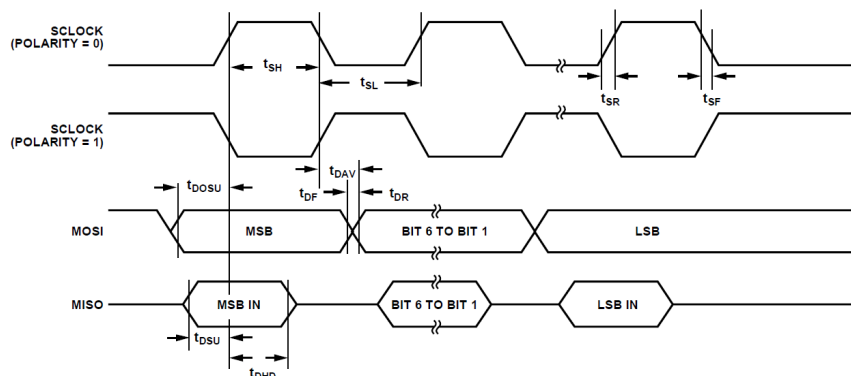


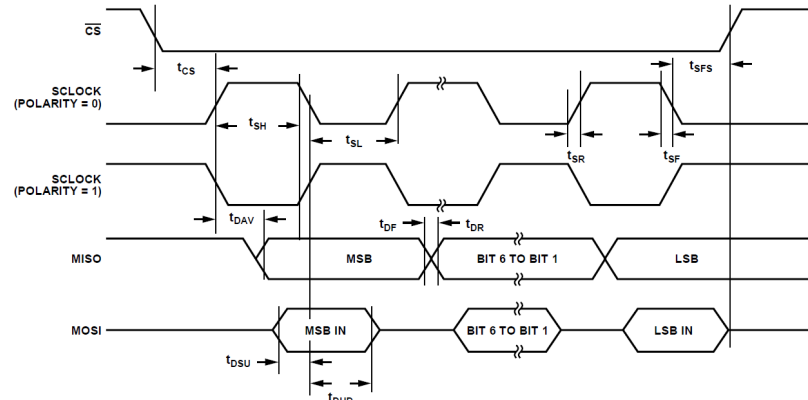
Рис. 21. Часові діаграми в режимі *Master*

SPI Slave Mode Timing (PHASE Mode = 1)

Parameter	Description	Min	Typ	Max	Unit
t_{CS}	CS to SCLOCK edge ¹	$2 \times t_{UCLK}$			ns
t_{SL}	SCLOCK low pulse width ²		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{SH}	SCLOCK high pulse width ²		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{DAV}	Data output valid after SCLOCK edge			$2 \times t_{HCLK} + 2 \times t_{UCLK}$	ns
t_{DSU}	Data input setup time before SCLOCK edge ¹	$1 \times t_{UCLK}$			ns
t_{DHD}	Data input hold time after SCLOCK edge ¹	$2 \times t_{UCLK}$			ns
t_{DF}	Data output fall time		5	12.5	ns
t_{DR}	Data output rise time		5	12.5	ns
t_{SR}	SCLOCK rise time		5	12.5	ns
t_{SF}	SCLOCK fall time		5	12.5	ns
t_{SFS}	CS high after SCLOCK edge	0			ns

¹ $t_{UCLK} = 23.9$ ns. It corresponds to the 41.78 MHz internal clock from the PLL before the clock divider.

² t_{HCLK} depends on the clock divider or CD bits in the PLLCON MMR, $t_{HCLK} = t_{UCLK}/2^{CD}$.



SPI Slave Mode Timing (PHASE Mode = 0)

Parameter	Description	Min	Typ	Max	Unit
t_{CS}	CS to SCLOCK edge ¹	$2 \times t_{UCLK}$			ns
t_{SL}	SCLOCK low pulse width ²		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{SH}	SCLOCK high pulse width ²		$(SPIDIV + 1) \times t_{HCLK}$		ns
t_{DAV}	Data output valid after SCLOCK edge			$2 \times t_{HCLK} + 2 \times t_{UCLK}$	ns
t_{DSU}	Data input setup time before SCLOCK edge ¹	$1 \times t_{UCLK}$			ns
t_{DHD}	Data input hold time after SCLOCK edge ¹	$2 \times t_{UCLK}$			ns
t_{DF}	Data output fall time		5	12.5	ns
t_{DR}	Data output rise time		5	12.5	ns
t_{SR}	SCLOCK rise time		5	12.5	ns
t_{SF}	SCLOCK fall time		5	12.5	ns
t_{DOCS}	Data output valid after CS edge			25	ns
t_{SFS}	CS high after SCLOCK edge	0			ns

¹ $t_{UCLK} = 23.9$ ns. It corresponds to the 41.78 MHz internal clock from the PLL before the clock divider.

² t_{HCLK} depends on the clock divider or CD bits in the PLLCON MMR, $t_{HCLK} = t_{UCLK}/2^{CD}$.

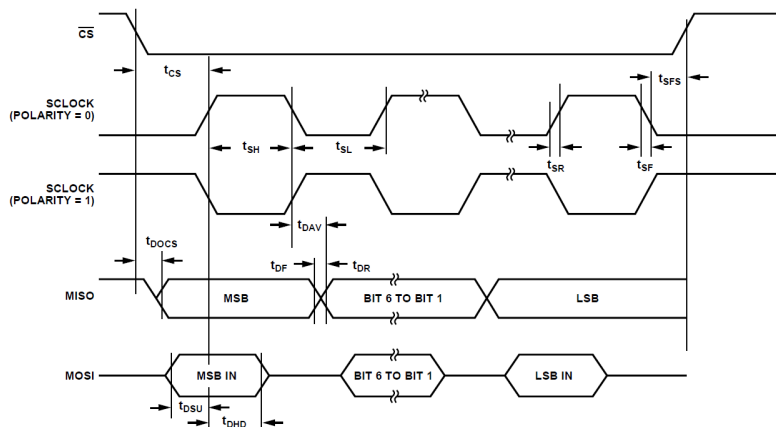


Рис. 22. Часові діаграми в режимі *Slave*

Мультиплексований порт SPM - SERIAL PORT MUX

SPM4	P1.4	RI0	SPICLK	PLAI[4]
SPM5	P1.5	DCD0	SPIMISO	PLAI[5]
SPM6	P1.6	DSR0	SPIMOSI	PLAI[6]
SPM7	P1.7	DTR0	SPICSL	PLAO[0]
SPICLK	(Serial Clock)		I/O Pin	
SPIMISO	(Master In, Slave Out)		I/O Pin	
SPIMOSI	(Master Out, Slave In)		I/O Pin	
SPICSL	Chip Select (CS)		Input Pin	

Регістри SPI

SPI Speed vs. Clock Divider Bits in Master Mode						
CD Bits	0	1	2	3	4	5
SPIDIV in hex	0x05	0x0B	0x17	0x2F	0x5F	0xBF
SPI speed in MHz	3.482	1.741	0.870	0.435	0.218	0.109

$$f_{SERIAL\ CLOCK} = \frac{f_{HCLK}}{2 \times (1 + SPIDIV)}$$

SPISTA Register

Name	Address	Default Value	Access
SPISTA	0xFFFF0A00	0x00	R

SPISTA is an 8-bit read-only status register.

SPISTA MMR Bit Designations

Bit	Description
7:6	Reserved.
5	SPIRX Data Register Overflow Status Bit. Set if SPIRX is overflowing. Cleared by reading SPIRX register.
4	SPIRX Data Register IRQ. Set automatically if Bit 3 or Bit 5 is set. Cleared by reading SPIRX register.
3	SPIRX Data Register Full Status Bit. Set automatically if valid data is present in the SPIRX register. Cleared by reading SPIRX register.
2	SPITX Data Register Underflow Status Bit. Set automatically if SPITX is underflowing. Cleared by writing in the SPITX register.
1	SPITX Data Register IRQ. Set automatically if Bit 0 is clear or Bit 2 is set. Cleared by writing in the SPITX register or if finished transmission disabling the SPI.
0	SPITX Data Register Empty Status Bit. Set by writing to SPITX to send data. This bit is set during transmission of data. Cleared when SPITX is empty.

SPIRX Register

Name	Address	Default Value	Access
SPIRX	0xFFFF0A04	0x00	R

SPIRX is an 8-bit read-only receive register.

SPITX Register

Name	Address	Default Value	Access
SPITX	0xFFFF0A08	0x00	W

SPITX is an 8-bit write-only transmit register.

SPIDIV Register

Name	Address	Default Value	Access
SPIDIV	0xFFFF0A0C	0x1B	R/W

SPIDIV is an 8-bit serial clock divider register.

SPICON Register

Name	Address	Default Value	Access
SPICON	0xFFFF0A10	0x0000	R/W

SPICON is a 16-bit control register.

SPICON MMR Bit Designations

Bit	Description
15:13	Reserved.
12	Continuous Transfer Enable. Set by user to enable continuous transfer. In master mode, the transfer continues until no valid data is available in the TX register. $\overline{CS}$ is asserted and remains asserted for the duration of each 8-bit serial transfer until TX is empty. Cleared by user to disable continuous transfer. Each transfer consists of a single 8-bit serial transfer. If valid data exists in the SPITX register, then a new transfer is initiated after a stall period.
11	Loopback Enable. Set by user to connect MISO to MOSI and test software. Cleared by user to be in normal mode.
10	Slave Output Enable. Set by user to enable the slave output. Cleared by user to disable slave output.
9	Slave Select Input Enable. Set by user in master mode to enable the output.
8	SPIRX Overflow Overwrite Enable. Set by user, the valid data in the RX register is overwritten by the new serial byte received. Cleared by user, the new serial byte received is discarded.
7	SPITX Underflow Mode. Set by user to transmit 0. Cleared by user to transmit the previous data.
6	Transfer and Interrupt Mode (Master Mode). Set by user to initiate transfer with a write to the SPITX register. Interrupt occurs when TX is empty. Cleared by user to initiate transfer with a read of the SPITX register. Interrupt occurs when RX is full.
5	LSB First Transfer Enable Bit. Set by user, the LSB is transmitted first. Cleared by user, the MSB is transmitted first.
4	Reserved. Should be set to 0.
3	Serial Clock Polarity Mode Bit. Set by user, the serial clock idles high. Cleared by user, the serial clock idles low.
2	Serial Clock Phase Mode Bit. Set by user, the serial clock pulses at the beginning of each serial bit transfer. Cleared by user, the serial clock pulses at the end of each serial bit transfer.
1	Master Mode Enable Bit. Set by user to enable master mode. Cleared by user to enable slave mode.
0	SPI Enable Bit. Set by user to enable the SPI. Cleared to disable the SPI.

Лабораторний стенд на базі EVAL-ADUC7128QSPZ



Рис. 23. Комплектація EVAL-ADUC7128QSPZ

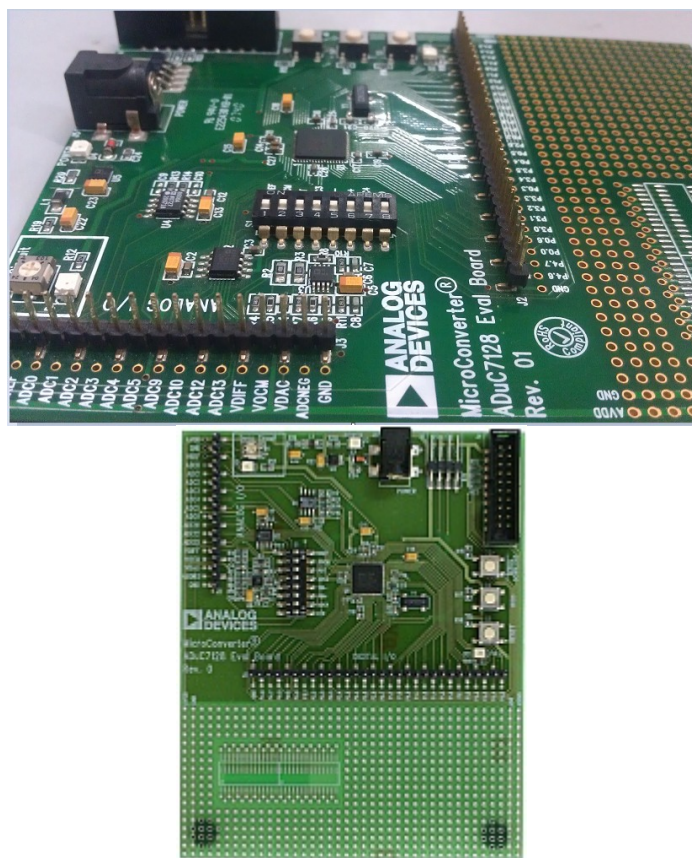


Рис. 24. Плата EVAL-ADUC7128QSPZ

Лістинги

```

/*****
File      : DACsine.c

Hardware   : ADuC7128 Rev C silicon

Description : Uses the DDS to output a sine wave on the DAC
*****/

#include "ADuC7128.h"
void delay (int length);
int main (void) {
    GP4DAT = 0x04000000;    // P4.2 configured as an output. LED is turned on

    PLLKEY1 = 0xAA;        // switch to external crystal
    PLLCON = 0x01;
    PLLKEY2 = 0x55;

    REFCON = 0x01;        // Reference must be decoupled for DAC operation

    DACCON = 0x3E;        // DAC set in DDS mode
    DDSCON = 0x28;        // DDS output enabled
    DDSFRQ = 0x08000000;   // Freq set to 652kHz

    while(1){
        GP4DAT ^= 0x00040000; // Complement P4.2
        delay(100000);
    }

}

void delay (int length)
{
    while (length >=0)
        length--;
}

/*****
File      : master.c

Hardware   : Applicable to ADuC7128 rev b or c silicon

Description : SPI master to demonstrate with slave.c or slave1.c
*****/

#include<ADuC7128.h>

unsigned char results[30] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06,
    0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10,
    0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x20,
    0x21, 0x22, 0x23, 0x24};

int main(void) {
    int i = 0;

    GP1CON = 0x22220000;    // configure SPI on SPM
    SPIDIV = 0xCC;          // set SPI clock 4096000/(2x(1+SPIDIV))
                            // 0xCC = 100kHz
    SPICON = 0x104B;        // enable SPI master in continuous transfer mode
                            // slave select will stay low during the all transmission

    for (i=0;i<30;i++)
    {
        SPITX = results[i]; // transmit command or any dummy data
        while ((SPISTA & 0x02) != 0x02) ; // wait for data received status bit
    }
    while (1){}
}

```



```

/*****
File      : slave.c
Hardware  : Applicable to ADuC7128 rev b or c silicon
           Currently targetting ADuC7128.
Description : SPI slave is to use with master.c or master1.c
           the slaves receives values from the master and
           keeps transmitting '0' as it is the default value at reset.
*****/

#include<ADuC7128.h>

int main(void) {
char i;
char received_data[30];

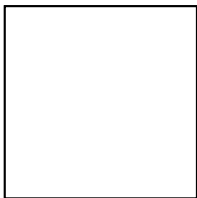
    GP1CON = 0x22220000;           // configure SPI on SPM
    SPICON = 0x1409;               // enable SPI slave mode

    for (i=0; i <30; i++) {
        while (!(SPISTA & 0x08)) ; // wait for data in the RX MMR
        received_data[i] = SPIRX;  // read data and clear bit 4 of SPISTA
    }
    while (1) {
        }
}

```

Література

1. ADUC7128_7129.pdf.
2. ADuC7128_ds.pdf.
3. ADuC7XXXGetStartedGuideV0 4.pdf
4. Конспект лекцій з дисципліни «Мікропроцесорні системи»



НАВЧАЛЬНЕ ВИДАННЯ

Дослідження цифрового синтезатора частоти в МПС

Методичні вказівки

до лабораторної роботи № 6 з курсу “ Мікропроцесорні системи ”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Укладач

Пуйда В. Я., к. т. н, доцент

Редактор

Комп’ютерне верстання

Здано у видавництво . Підписано до друку
Формат 70х100/16. Папір офсетний. Друк на різнографі
Умовн. друк. арк. Обл.-вид. арк..
Тираж прим. Зам..

Видавництво Національного університету “Львівська політехніка”
Реєстраційне свідоцтво ДК №751 від 27.12.2001 р.

Поліграфічний центр Видавництва
Національного університету “Львівська політехніка”

Вул.. Ф. Колесси, 2. Львів, 79000